AI ASSISTED CODING

ASSIGNMENT-16.1

Name: Harshitha Guda

H.T.No:2303A51102

Task 1 – Student Information System Schema

Task:

Design a database schema for a Student Information System and generate queries using AI.

Instructions:

• Tables: Students, Courses, Enrollments.

• Define primary keys, foreign keys, and relationships.

• Generate queries:

o Insert a new student record.

o Fetch all courses enrolled by a student.

o Count number of students in each course.

Expected Output:

• SQL CREATE TABLE statements and queries for student–course relationships.

```
#Create tables Students,Courses and Enrollments and then define primary keys foreign keys and relationships
CREATE TABLE Students (
    student_id INT PRIMARY KEY,
    name VARCHAR(50),
    age INT
);
CREATE TABLE Courses (
    course_id INT PRIMARY KEY,
    course_name VARCHAR(100)
);
CREATE TABLE Enrollments (
    enrollment_id INT PRIMARY KEY,
    student_id INT,
    course_id INT,
    FOREIGN KEY (student_id) REFERENCES Students(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

```sql
#Insert some data into the Students and Courses tables
INSERT INTO Students (student_id, name, age) VALUES (1, 'Alice', 20);
INSERT INTO Students (student_id, name, age) VALUES (2, 'Bob', 22);
INSERT INTO Courses (course_id, course_name) VALUES (1, 'Mathematics');
INSERT INTO Courses (course_id, course_name) VALUES (2, 'Physics');
#Enroll students in courses
INSERT INTO Enrollments (enrollment_id, student_id, course_id) VALUES (1, 1, 1);
INSERT INTO Enrollments (enrollment_id, student_id, course_id) VALUES (2, 1, 2);
INSERT INTO Enrollments (enrollment_id, student_id, course_id) VALUES (3, 2, 1);
INSERT INTO Enrollments (enrollment_id, student_id, course_id) VALUES (4, 2, 2);
#display all courses enrolled by a student
SELECT s.name AS student_name, c.course_name
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
JOIN Courses c ON e.course_id = c.course_id
WHERE s.student_id = 1;
#count the number of students enrolled in each course
SELECT c.course_name, COUNT(e.student_id) AS num_students
FROM Courses c
JOIN Enrollments e ON c.course_id = e.course_id
GROUP BY c.course_name;
```

SQL ▼ 1 / 1 1 - 4 of 4

student_name	course_name
Alice	Mathematics
Alice	Physics
Bob	Mathematics
Bob	Physics

SQL ▼ 1 / 1 1 - 2 of 2

course_name	num_students
Mathematics	2
Physics	2

Task 2 – Hospital Management Database

Task:

Create schema and queries for a Hospital Management System.

Instructions:

• Tables: Doctors, Patients, Appointments.

• Use AI to define constraints (unique IDs, valid dates).

• Generate queries:

o List all appointments for a specific doctor.

o Retrieve patient history by patient ID.

o Count total patients treated by each doctor.

Expected Output:

• Normalized schema and SQL queries with joins.

```sql
#Create  tables Doctors, Patients and Appointments and then define unique ids and valid dates and
#1.List all appointments for a specific doctor.2.Retrieve patient history by patient ID.
#3.Count total patients treated by each doctor. give queries with joins
CREATE TABLE Doctors (
    doctor_id INT PRIMARY KEY,
    name VARCHAR(50),
    specialty VARCHAR(50)
);
CREATE TABLE Patients (
    patient_id INT PRIMARY KEY,
    name VARCHAR(50),
    age INT
);
CREATE TABLE Appointments (
    appointment_id INT PRIMARY KEY,
    doctor_id INT,
    patient_id INT,
    appointment_date DATE,
    FOREIGN KEY (doctor_id) REFERENCES Doctors(doctor_id),
    FOREIGN KEY (patient_id) REFERENCES Patients(patient_id)
);
```

```sql
#Insert some data into the Doctors and Patients tables
INSERT INTO Doctors (doctor_id, name, specialty) VALUES (1, 'Dr. Smith', 'Cardiology');
INSERT INTO Doctors (doctor_id, name, specialty) VALUES (2, 'Dr. Johnson', 'Neurology');
INSERT INTO Patients (patient_id, name, age) VALUES (1, 'Alice', 30);
INSERT INTO Patients (patient_id, name, age) VALUES (2, 'Bob', 25);
INSERT INTO Patients (patient_id, name, age) VALUES (3, 'Charlie', 40);
#Schedule some appointments
INSERT INTO Appointments (appointment_id, doctor_id, patient_id, appointment_date) VALUES (1, 1, 1, '2023-10-15');
INSERT INTO Appointments (appointment_id, doctor_id, patient_id, appointment_date) VALUES (2, 1, 2, '2023-10-16');
INSERT INTO Appointments (appointment_id, doctor_id, patient_id, appointment_date) VALUES (3, 2, 3, '2023-10-17');
#1.List all appointments for a specific doctor (e.g., Dr. Smith)
SELECT a.appointment_date, p.name AS patient_name
FROM Appointments a
JOIN Patients p ON a.patient_id = p.patient_id
WHERE a.doctor_id = 1;
#2.Retrieve patient history by patient ID (e.g., patient ID 1)
SELECT a.appointment_date, d.name AS doctor_name, d.specialty
FROM Appointments a
JOIN Doctors d ON a.doctor_id = d.doctor_id
WHERE a.patient_id = 1;
#3.Count total patients treated by each doctor
SELECT d.name AS doctor_name, COUNT(a.patient_id) AS total_patients
FROM Doctors d
JOIN Appointments a ON d.doctor_id = a.doctor_id
GROUP BY d.name;
```

SQL ▼ 1 / 1 1 - 2 of 2

appointment_date	patient_name
2023-10-15	Alice
2023-10-16	Bob

SQL ▼ 1 / 1 1 - 1 of 1

appointment_date	doctor_name	specialty
2023-10-15	Dr. Smith	Cardiology

SQL ▼ 1 / 1 1 - 2 of 2

doctor_name	total_patients
Dr. Johnson	1
Dr. Smith	2

Task 3 – Library Database

Task:

Design schema for a Library Mana#gement System.

Instructions:

• Tables: Books, Members, Loans.

• Use AI to suggest indexing strategy for faster queries.

• Generate queries:

o Retrieve all books currently issued.

o Find overdue books (loan date > 30 days).

o Count number of books loaned by each member.

Expected Output:

• Schema + SQL queries demonstrating joins and conditions.

```sql
#create tables Books,Members and Loans and use indexing strategy to optimize queries then
#1.o Retrieve all books currently issued.2.Find overdue books (loan date > 30 days).
#3.Count number of books loaned by each member.
CREATE TABLE Books (
    book_id INT PRIMARY KEY,
    title VARCHAR(100),
    author VARCHAR(50)
);
CREATE TABLE Members (
    member_id INT PRIMARY KEY,
    name VARCHAR(50),
    membership_date DATE
);
CREATE TABLE Loans (
    loan_id INT PRIMARY KEY,
    book_id INT,
    member_id INT,
    loan_date DATE,
    return_date DATE,
    FOREIGN KEY (book_id) REFERENCES Books(book_id),
    FOREIGN KEY (member_id) REFERENCES Members(member_id)
);
```

```sql
#Create indexes on book_id and member_id in Loans table to optimize queries
CREATE INDEX idx_book_id ON Loans(book_id);
CREATE INDEX idx_member_id ON Loans(member_id);
#Insert some data into the Books and Members tables
INSERT INTO Books (book_id, title, author) VALUES (1, 'The Great Gatsby', 'F. Scott Fitzgerald');
INSERT INTO Books (book_id, title, author) VALUES (2, 'To Kill a Mockingbird', 'Harper Lee');
INSERT INTO Members (member_id, name, membership_date) VALUES (1, 'Alice',  '2023-01-01');
INSERT INTO Members (member_id, name, membership_date) VALUES (2, 'Bob', '2023-02-01');
#Schedule some loans
INSERT INTO Loans (loan_id, book_id, member_id, loan_date, return_date) VALUES (1, 1, 1, '2023-10-01', NULL);
INSERT INTO Loans (loan_id, book_id, member_id, loan_date, return_date) VALUES (2, 2, 2, '2023-09-01', '2023-10-01');
#1.Retrieve all books currently issued (i.e., return_date is NULL)
SELECT b.title, m.name AS member_name, l.loan_date
FROM Loans l
JOIN Books b ON l.book_id = b.book_id
JOIN Members m ON l.member_id = m.member_id
WHERE l.return_date IS NULL;
#2.Find overdue books (loan date > 30 days)
SELECT b.title, m.name AS member_name, l.loan_date
FROM Loans l
JOIN Books b ON l.book_id = b.book_id
JOIN Members m ON l.member_id = m.member_id
WHERE l.return_date IS NULL AND DATEDIFF(CURDATE(), l.loan_date) > 30;
#3.Count number of books loaned by each member
SELECT m.name AS member_name, COUNT(l.book_id) AS total_books_loaned
FROM Members m
JOIN Loans l ON m.member_id = l.member_id
GROUP BY m.name;
```

SQL ▼ 1 / 1 1 - 1 of 1

title	member_name	loan_date
The Great Gatsby	Alice	2023-10-01

SQL ▼ 1 / 1 1 - 2 of 2

member_name	total_books_loaned
Alice	1
Bob	1

Task 4 – Real-Time Application: Online Shopping Database

Scenario:

Design a database for an E-commerce platform.

Requirements:

• Tables: Users, Products, Orders, OrderDetails.

• Generate queries to:

o Retrieve all orders by a user.

o Find the most purchased product.

o Calculate total revenue in a given month.

• AI should also suggest normalization improvements and query optimization.

Expected Output: Complete schema with relationships + SQL queries for analytics.

```sql
#create tables Users,Products,Orders and Orderdetails and then
#1.Retrieve all orders by a user.2.Find the most purchased product.3.Calculate total revenue in a given month.
#then suggest normalization improvements and query optimizations
CREATE TABLE Users (
    user_id INT PRIMARY KEY,
    name VARCHAR(50),
    email VARCHAR(100)
);
CREATE TABLE Products (
    product_id INT PRIMARY KEY,
    product_name VARCHAR(100),
    price DECIMAL(10, 2)
);
CREATE TABLE Orders (
    order_id INT PRIMARY KEY,
    user_id INT,
    order_date DATE,
    FOREIGN KEY (user_id) REFERENCES Users(user_id)
);
CREATE TABLE OrderDetails (
    order_detail_id INT PRIMARY KEY,
    order_id INT,
    product_id INT,
    quantity INT,
    FOREIGN KEY (order_id) REFERENCES Orders(order_id),
    FOREIGN KEY (product_id) REFERENCES Products(product_id)
);
```

```sql
#Insert some data into the Users and Products tables
INSERT INTO Users (user_id, name, email) VALUES (1, 'Alice', 'alice@example.com');
INSERT INTO Products (product_id, product_name, price) VALUES (1, 'Laptop', 999.99);
INSERT INTO Products (product_id, product_name, price) VALUES (2, 'Mouse', 29.99);
#Schedule some orders
INSERT INTO Orders (order_id, user_id, order_date) VALUES (1, 1, '2023-10-01');
INSERT INTO OrderDetails (order_detail_id, order_id, product_id, quantity) VALUES (1, 1, 1, 1);
INSERT INTO OrderDetails (order_detail_id, order_id, product_id, quantity) VALUES (2, 1, 2, 2);
#1.Retrieve all orders by a user (e.g., user ID 1)
SELECT o.order_id, o.order_date, p.product_name, od.quantity
FROM Orders o
JOIN OrderDetails od ON o.order_id = od.order_id
JOIN Products p ON od.product_id = p.product_id
WHERE o.user_id = 1;
#2.Find the most purchased product
SELECT p.product_name, SUM(od.quantity) AS total_quantity
FROM OrderDetails od
JOIN Products p ON od.product_id = p.product_id
GROUP BY p.product_name
ORDER BY total_quantity DESC
LIMIT 1;
#3.Calculate total revenue in a given month
SELECT SUM(p.price * od.quantity) AS total_revenue
FROM OrderDetails od
JOIN Products p ON od.product_id = p.product_id
JOIN Orders o ON od.order_id = o.order_id
WHERE YEAR(o.order_date) = 2023 AND MONTH(o.order_date) = 10;
#Normalization improvements:
#1. The current schema is already normalized to 3NF, but we can further optimize by creating a separate table for categories if products belong to different categories.
#2. We can also consider adding indexes on frequently queried columns such as user_id in Orders and product_id in OrderDetails to improve query performance.
```

SQL ▼ 1 / 1 1 - 2 of 2

order_id	order_date	product_name	quantity
1	2023-10-01	Laptop	1
1	2023-10-01	Mouse	2

SQL ▼ 1 / 1 1 - 1 of 1

product_name	total_quantity
Mouse	2

Task 5 – University Examination Database

Design a database schema for a University Examination System and generate SQL queries using AI.

Instructions:

• Tables: Students, Subjects, Exams, Results.

• Define primary keys, foreign keys, and referential integrity constraints.

• Generate queries:

o Insert examination results for a student.

o Retrieve subject-wise marks for a student.

o Calculate average marks for each subject.

Expected Output:

• Normalized schema and SQL queries involving aggregation and joins.

```sql
#create tables Student_names ,subjects,exams and results and then define primary keys and foreign keys and also referential integrity constraints then
#1.Insert examination results for a student.2.Retrieve subject-wise marks for a student.3.Calculate average marks for each subject.
CREATE TABLE Student_names (
    student_id INT PRIMARY KEY,
    name VARCHAR(50),
    age INT
);
CREATE TABLE Subjects (
    subject_id INT PRIMARY KEY,
    subject_name VARCHAR(50)
);
CREATE TABLE Exams (
    exam_id INT PRIMARY KEY,
    subject_id INT,
    exam_date DATE,
    FOREIGN KEY (subject_id) REFERENCES Subjects(subject_id)
);
CREATE TABLE Results (
    result_id INT PRIMARY KEY,
    student_id INT,
    exam_id INT,
    marks INT,
    FOREIGN KEY (student_id) REFERENCES Student_names(student_id),
    FOREIGN KEY (exam_id) REFERENCES Exams(exam_id)
);
```

```sql
#Insert some data into the Student_names and Subjects tables
INSERT INTO Student_names (student_id, name, age) VALUES (1, 'Alice', 20);
INSERT INTO Subjects (subject_id, subject_name) VALUES (1, 'Mathematics');
INSERT INTO Subjects (subject_id, subject_name) VALUES (2, 'Physics');
#Schedule some exams
INSERT INTO Exams (exam_id, subject_id, exam_date) VALUES (1, 1, '2023-10-01');
INSERT INTO Exams (exam_id, subject_id, exam_date) VALUES (2, 2, '2023-10-02');
#1.insert examination results for a student (e.g., student ID 1)
INSERT INTO Results (result_id, student_id, exam_id, marks) VALUES (1, 1, 1, 85);
INSERT INTO Results (result_id, student_id, exam_id, marks) VALUES (2, 1, 2, 90);
#2.Retrieve subject-wise marks for a student (e.g., student ID 1)
SELECT s.subject_name, r.marks
FROM Results r
JOIN Exams e ON r.exam_id = e.exam_id
JOIN Subjects s ON e.subject_id = s.subject_id
WHERE r.student_id = 1;
#3.Calculate average marks for each subject
SELECT s.subject_name, AVG(r.marks) AS average_marks
FROM Results r
JOIN Exams e ON r.exam_id = e.exam_id
JOIN Subjects s ON e.subject_id = s.subject_id
GROUP BY s.subject_name;
```

SQL ▼ 1 / 1 1 - 2 of 2

subject_name	marks
Mathematics	85
Physics	90

SQL ▼ 1 / 1 1 - 2 of 2

subject_name	average_marks
Mathematics	85.0
Physics	90.0